Portfolio-Conditioned Liquidity Impact in Corporate Bonds
A Multi-View Differential Graph Transformer

Oleg Roshka *

Abstract

We study whether a neural model can learn the *portfolio-conditioned component* of transaction costs in corporate bonds. Let y denote the realized premium (bps), e.g., slippage vs. mid or executed half-spread, and let π^{ref} be a baseline in bps obtained from a vendor or a pre-trade proxy. We decompose:

$$y = \pi^{\mathrm{ref}}(x_{\mathrm{trade}}, x_{\mathrm{market}}) + \Delta(b, P_t, M_t) + \varepsilon,$$

where $\Delta(b, P_t, M_t)$ is the (unknown) adjustment conditioned on bond b, the contemporaneous basket P_t, and market state M_t. We introduce the *Multi-View Differential Graph Transformer* (MV-DGT): a graph-transformer architecture with relation-specific message passing, *differential view fusion* (non-structural views contribute only their deviation from structural), strict leave-one-out (LOO) portfolio prototypes weighted by |DV01|, and optional *within-portfolio self-attention* (2 layers, 4 heads) with residual gating. On **simulated data** with a planted portfolio-conditioned signal, the model achieves strong parity on unseen portfolios: line-level residuals have MAE $\approx$3.3 bps, RMSE $\approx$4.1 bps, $\rho\approx$0.95; basket means are tighter (MAE $\approx$1.0 bps, RMSE $\approx$1.24 bps). We document a consistent with-portfolio vs. no-portfolio shift of $\sim$2 bps, demonstrating that MV-DGT can recover a stable portfolio-conditioned effect under controlled conditions—establishing *architectural feasibility* for this class of problems, pending validation on real transaction data.

1 Introduction

Vendor liquidity proxies are widely used to set bid–offer spreads, yet they are computed per security and ignore how a *live portfolio* changes effective liquidity. Portfolio trading creates cross-impact through issuer/sector concentration and DV01 geometry. We ask: *can a neural architecture learn a* ***portfolio-conditioned*** *adjustment* $\Delta(b, P)$*, and if so, what design choices enable robust recovery?*

To answer this, we construct a controlled experimental setting using *simulated* corporate bond data with a planted portfolio-conditioned signal. This allows us to (i) verify that the model recovers the true effect via ablation, and (ii) isolate architectural contributions from data-quality confounds. We emphasize that results on synthetic data do not guarantee identical performance on real transactions; rather, they establish *architectural feasibility* and provide a benchmark for future work with proprietary or TRACE-derived labels.

Contributions.

(i) A multi-view graph architecture that fuses structural relations (issuer, sector, rating, tenor), reconstructed co-trade links, and correlation/MI priors via *differential fusion*—non-structural views contribute only their deviation from structural, with learnable per-view gates.

(ii) Strict LOO, side-agnostic portfolio prototypes using absolute DV01 weights, plus an optional *within-portfolio Transformer encoder* (2 layers, 4 heads) with residual gating for cross-item interactions.

*Project contributor; email: `oleg.roshka@proton.me`

(iii) A complete training and serving pipeline with market standardization, orientation-sign correction, and portfolio-aware scoring.

(iv) Comprehensive diagnostics demonstrating parity on unseen portfolios and a stable *portfolio-conditioned liquidity pressure* of approximately 2 bps.

2 Related Work

We build on graph neural networks and attention [1, 2, 3], permutation-invariant set modeling [4], and graph transformers [5]. For differential graph transformers in markets, see the CS224W write-up [9]. On microstructure, we connect to cross-impact and price-impact views of liquidity [6, 7]; MI background follows Cover–Thomas [8].

3 Mathematical formulation and model

Notation. We summarize recurring symbols used throughout. Let $\mathcal{B}$ denote the bond universe; $b \in \mathcal{B}$ indexes a bond. t indexes a trading day; $P_t \subseteq \mathcal{B}$ is the multiset of bonds in the contemporaneous basket. $M_t \in \mathbb{R}^m$ is the market-state vector (e.g., MOVE, VIX, OAS levels). $h_i^{(\ell)} \in \mathbb{R}^d$ is the hidden representation of node i after layer ℓ; d denotes hidden dimension and L the total number of GNN layers (here $L = 2$). $\mathcal{N}^{(v)}(i)$ is the neighbor set of node i under view v. $\sigma(\cdot)$ denotes the sigmoid function $\sigma(x) = 1/(1 + e^{-x})$. $z_b \in \mathbb{R}^d$ is the final anchor representation fed to the prediction head.

3.1 Targets and residualization

Let $b \in \mathcal{B}$ index a bond, t a trading day, and P_t a multiset of bonds in the contemporaneous basket (portfolio trade, list RFQ, or desk inventory snapshot). We write the realized premium $y_{b,t}$ in bps as

$$y_{b,t} \;=\; \pi^{\text{ref}}(x_{\text{trade},b,t}, x_{\text{market},t}) \;+\; \Delta(b, P_t, M_t) \;+\; \varepsilon_{b,t}. \tag{1}$$

The baseline π^{ref} absorbs provider mapping and deterministic trade/market terms;[1] the *portfolio-conditioned liquidity pressure* is Δ. We train on the residual $r_{b,t} = y_{b,t} - \pi^{\text{ref}}(\cdot)$ and learn a function f_θ such that

$$\widehat{\Delta}(b, P_t, M_t) \;=\; f_\theta(b, P_t, M_t) \;\approx\; \mathbb{E}[r_{b,t} \mid b, P_t, M_t]\,. \tag{2}$$

3.2 Graph views

We represent the bond universe as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where each node $i \in \mathcal{V}$ corresponds to a bond with feature vector $x_i \in \mathbb{R}^F$ encoding categorical attributes (rating, tenor bucket, sector, currency) and optional numeric descriptors. We partition the edge set into four relation *views*:

- `struct`: stable economic substitutability (issuer, sector, rating proximity, tenor bucket);
- `port`: reconstructed co-trade links accumulated from historical portfolio trades;
- `corr_global`: absolute Pearson correlation on a global lookback window;
- `corr_local`: rolling local-window correlation/MI prior.

For each view v we have a weighted, undirected edge set $\mathcal{E}^{(v)}$ with normalized weights $w_{ij}^{(v)} \in [0, 1]$, quantile-pruned and Top-K filtered per node for sparsity.

[1] If evaluated prices or vendor liquidity scores are available, one can fit a simple provider-specific monotone mapping g_p to express the baseline in bps. In this prototype the baseline is taken as given.

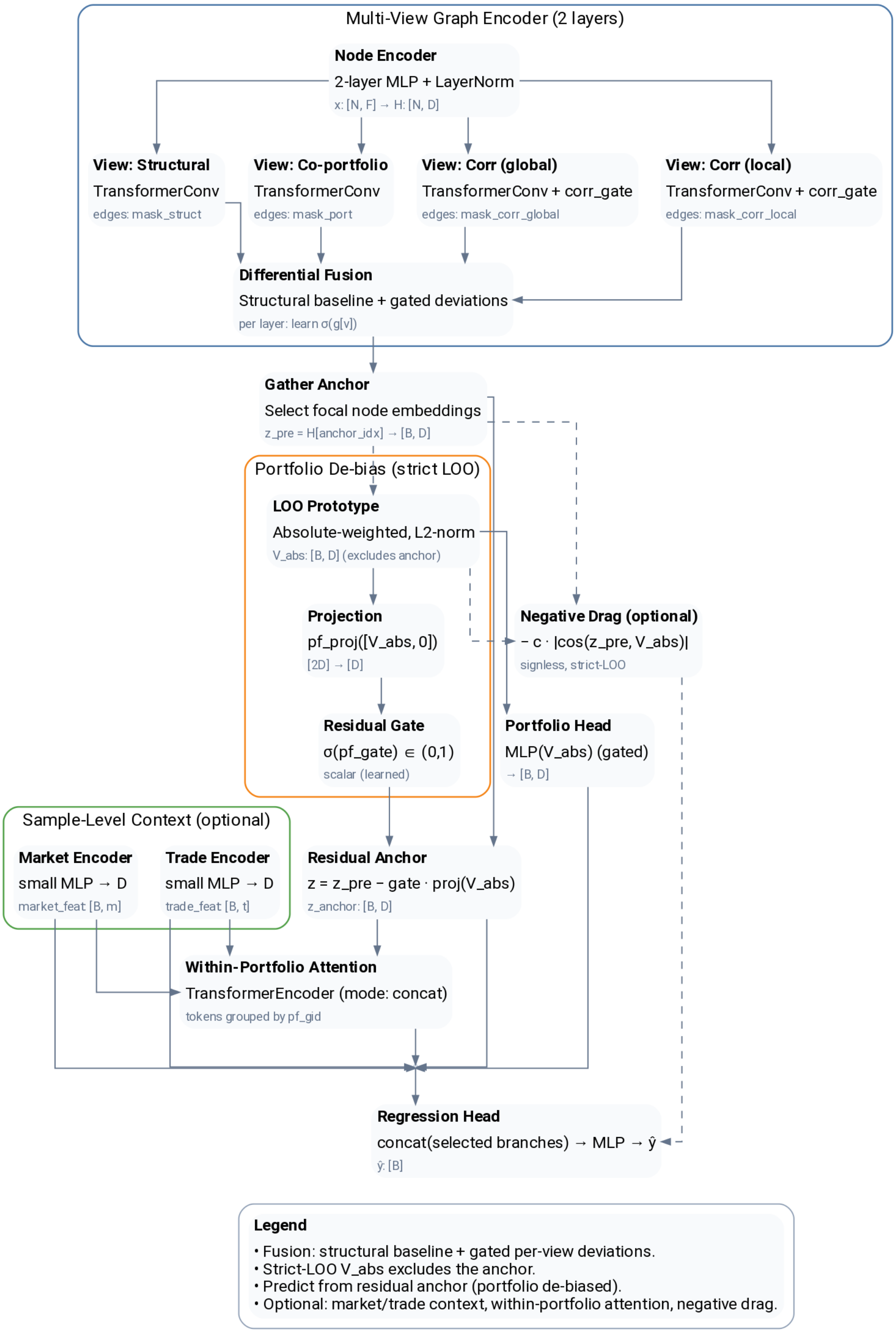


Figure 1: **MV-DGT end-to-end architecture (high level).** Node features are embedded and passed through multi-view graph Transformer message passing. Views include structural relations (issuer/sector/rating/tenor, etc.), portfolio/co-trade links, and correlation priors. View outputs are combined via *differential fusion* relative to the structural baseline (Eq. 5). A strict leave-one-out portfolio prototype and optional within-portfolio attention provide portfolio conditioning before the regression head predicts the residual adjustment $\widehat{\Delta}$.

3.3 Relation-specific Transformer message passing

We embed node features via a two-layer MLP with layer normalization:

$$h_i^{(0)} = \mathrm{LayerNorm}\big(\mathrm{MLP}(x_i)\big) \in \mathbb{R}^d. \tag{3}$$

At each layer $\ell \in \{1, \ldots, L\}$ and view v, we apply multi-head attention (TransformerConv) to compute a message:

$$m_i^{(\ell,v)} = \sum_{j \in \mathcal{N}^{(v)}(i)} \alpha_{ij}^{(\ell,v)} W_V^{(\ell,v)} h_j^{(\ell-1)}, \quad \alpha_{ij}^{(\ell,v)} \propto w_{ij}^{(v)} \exp\left(\frac{\langle W_Q^{(\ell,v)} h_i^{(\ell-1)}, W_K^{(\ell,v)} h_j^{(\ell-1)} \rangle}{\sqrt{d}}\right), \tag{4}$$

where $W_Q^{(\ell,v)}, W_K^{(\ell,v)}, W_V^{(\ell,v)} \in \mathbb{R}^{d\times d}$ are learned per view and layer. The edge weight $w_{ij}^{(v)}$ multiplicatively modulates attention logits, allowing the graph structure to bias the softmax.

Differential fusion. Rather than sum messages from all views directly, we treat the `struct` view as a *baseline* and let other views contribute only their *deviations*:

$$h_i^{(\ell)} = h_i^{(\ell-1)} + \sigma(\beta_{\texttt{struct}}^{(\ell)})\, m_i^{(\ell,\texttt{struct})} + \sum_{v \in \{\texttt{port},\texttt{corr\_global},\texttt{corr\_local}\}} \sigma(\beta_v^{(\ell)})\big(m_i^{(\ell,v)} - m_i^{(\ell,\texttt{struct})}\big), \tag{5}$$

where $\beta_v^{(\ell)} \in \mathbb{R}$ are learnable scalar gate logits initialized to zero (so $\sigma(\beta) = 0.5$ at initialization). This formulation bounds each view's contribution, reduces scale mismatch across heterogeneous relation types, and yields interpretable gate magnitudes. A separate learned gate $\sigma(\beta_{\texttt{corr}})$ further down-weights the aggregate correlation-view contribution, reflecting their noisier nature.

3.4 Portfolio prototype and within-portfolio attention

For an anchor bond $b \in P_t$, we construct a *strict leave-one-out* (LOO) portfolio prototype from the remaining basket members $P_t \setminus \{b\}$. Define absolute DV01-weighted coefficients:

$$\omega_j^{\mathrm{abs}} = \frac{|\mathrm{DV01}_j|}{\sum_{k \in P_t \setminus \{b\}} |\mathrm{DV01}_k|}, \quad j \in P_t \setminus \{b\}, \tag{6}$$

which sum to unity and are *side-agnostic* (ignoring buy/sell direction). The signless LOO prototype is the DV01-weighted average of final-layer embeddings:

$$\bar{h}_{P_t \setminus b}^{\mathrm{abs}} = \sum_{j \in P_t \setminus \{b\}} \omega_j^{\mathrm{abs}}\, h_j^{(L)}, \tag{7}$$

which we L2-normalize before fusion to ensure scale invariance: $\hat{h}^{\mathrm{abs}} = \bar{h}^{\mathrm{abs}} / \|\bar{h}^{\mathrm{abs}}\|_2$.

Within-portfolio Transformer encoder. Optionally, we run a standard Transformer encoder (2 layers, 4 heads) over tokens $\{t_b, t_j : j \in P_t \setminus \{b\}\}$, where each token is the corresponding node embedding (optionally concatenated with trade features). The attended anchor token $\tilde{t}_b$ captures cross-item interactions while preserving permutation invariance.

Final anchor representation. The anchor's representation z_b fuses three sources via gated residuals:

$$z_b = h_b^{(L)} + \sigma(\gamma_{\texttt{pf}}) \cdot \mathrm{MLP}(\hat{h}_{P_t \setminus b}^{\mathrm{abs}}) + \sigma(\gamma_{\texttt{attn}}) \cdot \tilde{t}_b, \tag{8}$$

where $\gamma_{\texttt{pf}}, \gamma_{\texttt{attn}} \in \mathbb{R}$ are learnable gate logits (initialized to zero). This construction guarantees permutation invariance in P_t, enforces strict LOO (the anchor never conditions on itself), and enables learned cross-item effects.

3.5 Regression head and loss

Feature encoding. Market features M_t are z-scored using train-split statistics and multiplied by a single *orientation sign* (inferred via ridge regression on train data) to remove arbitrary sign conventions. Trade features—side indicator $s_b \in \{-1, +1\}$ and log(size)—are standardized likewise. Let $\tilde{m}_t = \mathrm{MLP}_{\mathrm{mkt}}(M_t) \in \mathbb{R}^d$ and $\tilde{u}_b = \mathrm{MLP}_{\mathrm{trade}}([s_b, \log(\mathrm{size}_b)]) \in \mathbb{R}^d$ denote the encoded market and trade features.

Prediction head. The regression head concatenates anchor, market, and trade representations and applies a two-layer MLP:

$$\widehat{r}_{b,t} = \mathrm{MLP}_{\mathrm{head}}\big([z_b \,\|\, \tilde{m}_t \,\|\, \tilde{u}_b]\big) \in \mathbb{R}, \tag{9}$$

outputting the predicted residual in bps.

Loss function. We minimize a composite objective:

$$\mathcal{L} = \sum_{(b,t)} \kappa_{b,t} \cdot \mathrm{SmoothL1}\big(\widehat{r}_{b,t}, r_{b,t}\big) + \lambda_{\mathrm{sim}} \mathcal{L}_{\mathrm{sim}}, \tag{10}$$

where SmoothL1 (Huber loss with $\delta = 1$) is robust to outliers, $\kappa_{b,t} > 1$ up-weights portfolio rows relative to singleton trades, and $\mathcal{L}_{\mathrm{sim}}$ is a mild auxiliary term encouraging portfolio-level prediction consistency ($\lambda_{\mathrm{sim}} = 0.1$).

4 Data and graph construction

Structural relations. We connect bonds by issuer, sector, adjacent ratings, curve bucket, and (optionally) currency. These encode stable economic substitutability: issuer peers (same parent), sector co-membership, one-notch rating proximity, tenor bucket proximity, and same currency. Edges are quantile-pruned and Top-K per node for sparsity control.[2]

Co-trade relations. We identify groups of bonds executed together on the same date (portfolio/list trades) and accumulate pairwise co-trade scores that combine (i) recency decay, (ii) exposure coherence via the smaller absolute DV01, and (iii) bond similarity (sector/rating/tenor). Pairs are then split by sign of the sidedness product into:

- **COTRADE_CO (co-directional):** both lines in the pair are bought or sold together (positive sign); weight equals the aggregated magnitude of co-trading.
- **COTRADE_X (cross-directional):** one line is bought while the other is sold (negative sign); weight equals the aggregated magnitude of cross-directional co-trading.

We apply an absolute-value quantile threshold to keep only informative edges and retain per-node Top-K for each slab.

Correlation/MI priors. From per-bond daily "surprise" series (demeaned and z-scored price residuals) we compute absolute Pearson correlations and plug-in mutual information after quantile discretization; both are constructed on global and rolling local windows and kept Top-K per node. These form the `corr_global`/`corr_local` views; a learned gate down-weights their aggregate contribution.

[2]Exact Top-K caps per relation and pruning order follow the featurizer; see Appendix B for a compact summary.

Portfolio and market context. For each basket we retain: the list of constituent bonds, their *absolute* and *signed* DV01 fractions (unit-sum within the basket), and the basket length/offsets required to process ragged sets efficiently. Dates are aligned to a business-day index and we build market features from MOVE, VIX, and IG/HY OAS levels and changes (with limited forward-fill and rolling z-scores).

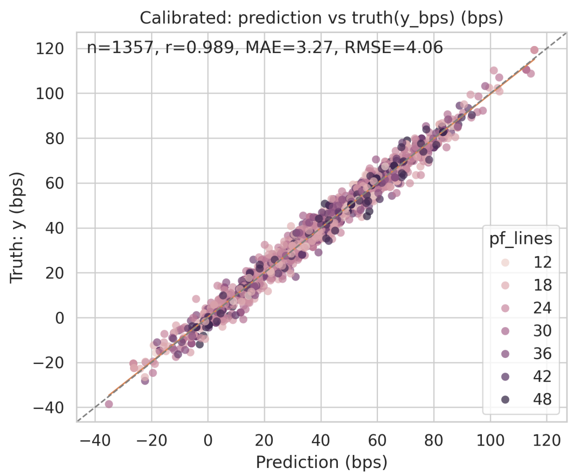
5 Training configuration and evaluation protocol

We train for 30 epochs with AdamW (lr 0.005, weight decay 10^{-4}), batch size 512, OneCycleLR, seed 42, on GPU. Hidden size 192; Transformer heads 4; dropout 0.1. *Within-portfolio self/cross-attention is enabled* (2 layers, 4 heads) with residual gating; the portfolio head is on (hidden 128). Training uses signless portfolio weights, in-batch dynamic portfolio context, and an auxiliary portfolio-similarity weight 0.1. Chronological splits are used; metrics are reported in bps on held-out lines and on basket means.

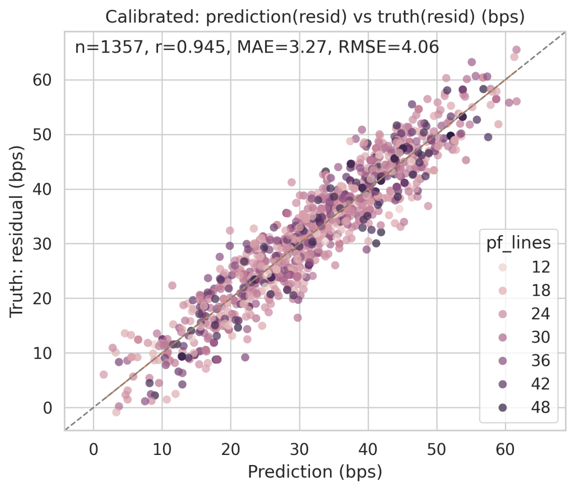
6 Results

6.1 Unseen portfolios: line-level parity

Figures 2a and 2b show strong parity for 1,357 unseen lines on y and residuals, respectively. Calibrated and "no-pf" variants (Appendix D) confirm that the portfolio path meaningfully shifts predictions without harming calibration.



(a) **Line-level parity on** y. $\rho = 0.989$, MAE≈3.27 bps, RMSE≈4.06 bps. *Interpretation:* points hug the 45° line across the full range, indicating minimal bias and tight absolute error—i.e., the model reproduces realized bps even away from the median.



(b) **Line-level residual parity**. $\rho = 0.945$, MAE≈3.27 bps, RMSE≈4.06 bps. *Interpretation:* after removing π^{ref}, predictions still track realized residuals with near-unit slope—evidence that the model learns the portfolio-conditioned component rather than memorizing the baseline.

Figure 2: **Unseen portfolios: line-level parity**.

6.2 Portfolio-conditioned liquidity pressure

Across the same 1,357 items, the distribution of $\Delta = \widehat{r}\,(\text{with pf}) - \widehat{r}\,(\text{no pf})$ is tight with median 2.04 bps and IQR 0.08 bps (Fig. 3). This quantifies the learned *portfolio-conditioned liquidity pressure*: relative to scoring a bond in isolation, portfolio context increases predicted residuals by about 2 bps on average.

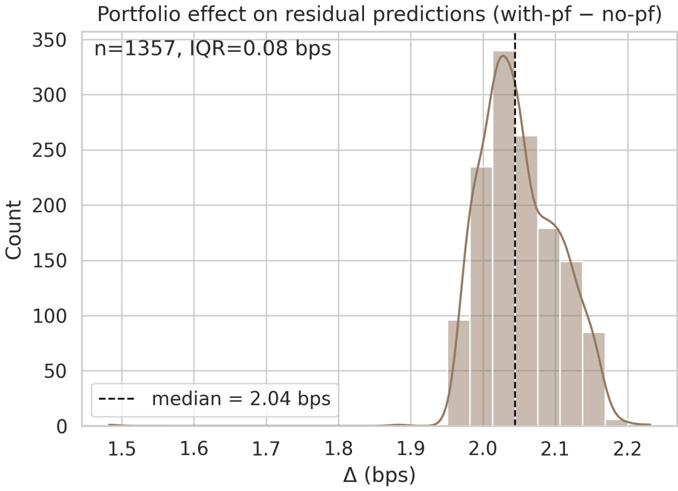
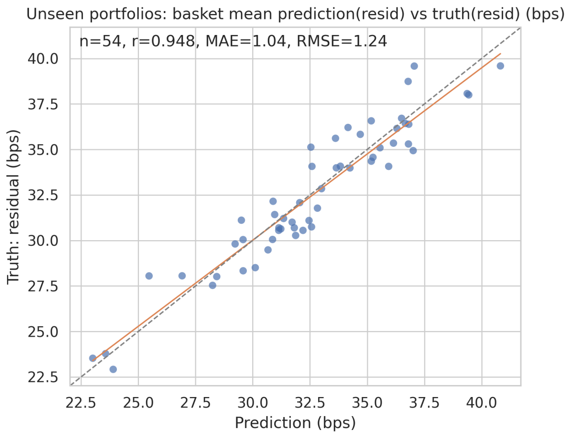


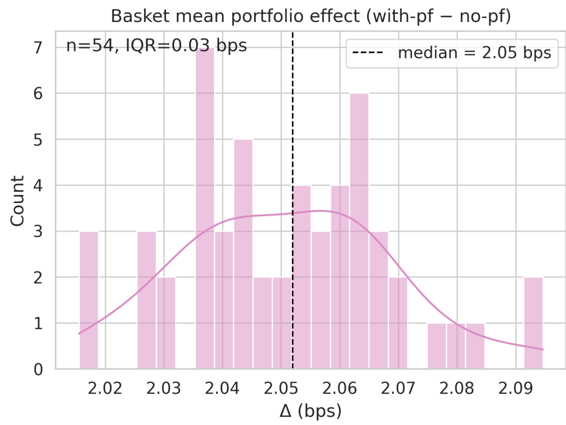
Figure 3: **Portfolio effect on residuals (with-pf − no-pf)**: median 2.04 bps, n=1,357. *Interpretation:* a narrow, unimodal shift implies the LOO portfolio path contributes a stable additive adjustment rather than sporadic, idiosyncratic moves.

6.3 Basket-level aggregation

Basket means (n=54) show very tight parity (MAE ≈1.0 bps, RMSE ≈1.24 bps, $\rho \approx 0.95$) and a consistent with-pf minus no-pf shift around 2 bps (Fig. 4).



(a) Basket mean residual parity. *Interpretation:* averaging within baskets cancels idiosyncratic noise; the slope and tight spread around the 45° line show that portfolio-level aggregation is reliably captured.



(b) Basket mean with-pf minus no-pf. *Interpretation:* the same ∼2 bps shift persists after aggregation, confirming that the pressure is systematic across names rather than a single-name artifact.

Figure 4: **Unseen portfolios: basket means**.

6.4 Diagnostics: side flip and size elasticity

Warm scenario probes are economically sensible: *side flip* (C–A) centers near −15.9 bps; *size elasticity* (B–A) near +1.07 bps. Cold probes have larger magnitudes/dispersion (see Appendix D).

6.5 Ablation

Turning off the portfolio path collapses the drift (Fig. 5); single-line parity vs. the baseline shows a small, consistent residual offset (median 1.62 bps, Appendix D).

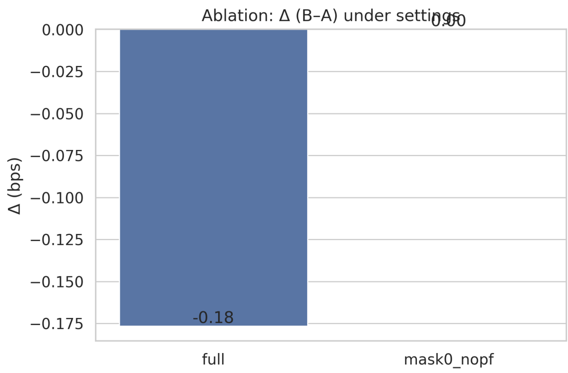


Figure 5: **Ablation**: line-level Δ(B–A) under full model vs. masked portfolio path. *Interpretation:* when portfolio context is suppressed, the effect shrinks to ≈ 0; hence the observed drift is genuinely portfolio-driven.

7 Operational use cases

1. **Portfolio-conditioned liquidity pressure monitoring.** If a portfolio snapshot (or basket RFQ composition) is available, summarize $\widehat{\Delta}$ across holdings (level and day-over-day change) to visualize current *pressure* on the book.

2. **Pre-trade basket design.** What-if add/swap/remove; search over issuer/sector/curve geometry to minimize predicted pressure for a target DV01 or client mandate.

3. **Quote/risk add-ons for baskets.** For list-style inquiries, adjust per-line quotes by $\kappa\,\widehat{\Delta}$ and surface lines whose inclusion most reduces basket pressure.

4. **Inventory unwind planning.** Sequence unwinds across names/sizes to reduce cumulative pressure; greedy or local-search over $\widehat{\Delta}$.

5. **Liquidity stress what-ifs.** Evaluate how $\widehat{\Delta}$ changes under MOVE/VIX/OAS shocks as a liquidity add-on to scenario analysis.

8 Limitations and discussion

Simulated data. All experiments in this paper are conducted on *synthetic* data generated according to the DGP in Appendix B. While the simulator is designed to capture stylized features of corporate bond markets—including portfolio-conditioned effects, market regimes, and realistic trade/bond attributes—it necessarily simplifies many real-world complexities: dealer inventory constraints, information asymmetry, client-specific pricing, intraday dynamics, and the full heterogeneity of liquidity across credit quality and market conditions. Consequently, the reported metrics (MAE $\approx$3.3 bps, $\rho \approx 0.95$, portfolio shift $\sim$2 bps) should be interpreted as demonstrating that the architecture *can* recover a planted portfolio-conditioned signal under controlled conditions—not as evidence that identical performance will transfer to real TRACE or proprietary transaction data.

Architectural validity. Despite the synthetic setting, we believe the study contributes meaningfully: (i) it establishes that differential multi-view fusion and strict LOO prototypes are architecturally sound for portfolio-conditioned regression; (ii) ablations confirm that the learned $\sim$2 bps shift genuinely originates from the portfolio path rather than confounders; (iii) the pipeline (graph construction, preprocessing, training loop) is fully implemented and can be adapted to real labels when available.

Other limitations. Portfolio and correlation views are reconstructed from co-trade history and can be misspecified in sparse regimes. Uncertainty quantification and production safeguards (drift detection, guardrails) are not yet integrated. Future work includes validation on real portfolio-trade labels, richer microstructure features, market-aware portfolio attention, and calibrated prediction intervals.

9 Conclusion

We introduced MV-DGT, a multi-view differential graph transformer for learning portfolio-conditioned liquidity adjustments in corporate bonds. On *simulated* data with a planted portfolio-conditioned signal, the model recovers a stable $\sim$2 bps shift attributable to the portfolio path, with strong line-level and basket-level parity on held-out portfolios.

These results demonstrate *architectural feasibility*: differential view fusion, strict LOO prototypes, and within-portfolio attention together enable a neural model to isolate portfolio-specific effects from bond-level and market-level variation. Whether—and to what extent—such effects exist in real transaction data remains an open empirical question. Nonetheless, the architecture, pipeline, and ablation methodology developed here provide a concrete starting point for practitioners seeking to quantify portfolio-conditioned liquidity pressure once suitable labeled data become available.

A Formal details (views, gates, preprocessing)

Differential view fusion. Let $\Phi^{(\ell,v)}(H)$ denote the TransformerConv message aggregation on view v at layer ℓ, where $H \in \mathbb{R}^{N\times d}$ is the current node embedding matrix. Gates $\sigma(\beta_v^{(\ell)})$ are learned scalars (one per view per layer), initialized at $\beta_v^{(\ell)} = 0$ so that $\sigma(0) = 0.5$. A separate global correlation gate $\sigma(\beta_{\texttt{corr}})$, initialized at -1.0 (yielding $\sigma(-1) \approx 0.27$), multiplicatively scales correlation-view edge weights before attention. Residual fusion is always computed with respect to the structural view, ensuring bounded contribution and interpretable gate magnitudes.

Portfolio prototypes and LOO invariance. For each basket P_t we store: (i) the list of member node indices, (ii) their absolute DV01 fractions ω_j^{abs} (unit-sum within the basket), and (iii) signed DV01 fractions for optional directional analyses. Strict LOO removes the anchor from its own context when computing $\bar{h}^{\text{abs}}_{P_t\setminus b}$. L2-normalizing the prototype ensures scale invariance to portfolio size and total exposure.

Market preprocessing and trade scalers. Market features M_t (MOVE, VIX, IG/HY OAS levels and daily changes) are z-scored using train-split statistics. A single orientation sign vector, computed via ridge regression of targets on market features over train dates, removes arbitrary sign conventions. Trade features (side sign $s \in \{-1, +1\}$ and log(size)) are standardized by train mean and standard deviation. Portfolio rows receive up-weighting factor $\kappa > 1$ in the loss to emphasize basket-level learning.

A.1 Strict leave-one-out (LOO) portfolio prototypes (diagram)

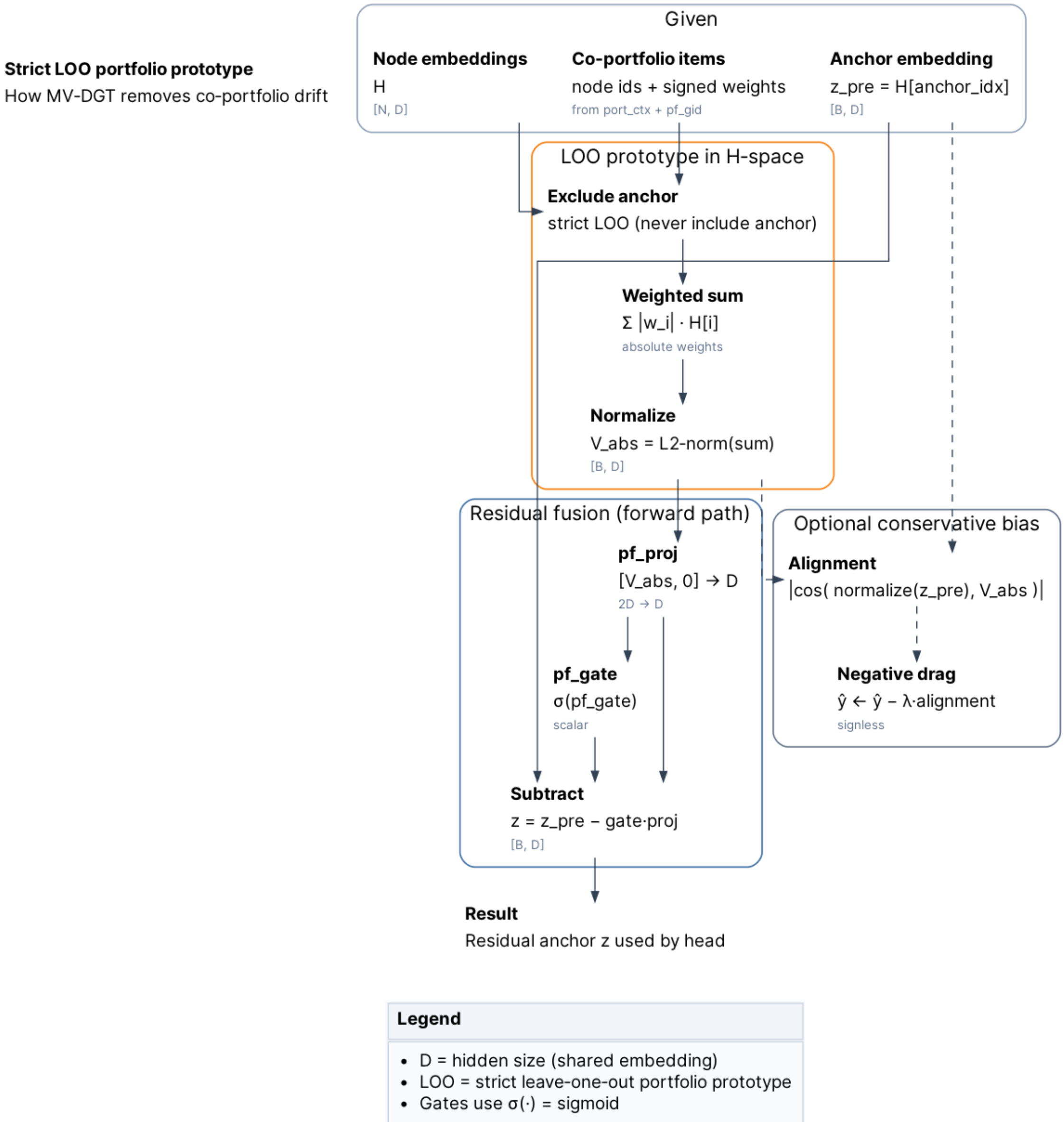


Figure 6: **Strict LOO portfolio prototype (conceptual).** For each anchor bond $b \in P_t$, the portfolio context $\bar{h}^{\text{abs}}_{P_t \setminus b}$ is computed from $P_t \setminus \{b\}$ to prevent leakage of the anchor into its own conditioning signal. Using absolute DV01 weights ω_j^{abs} yields a side-agnostic prototype. Optional within-portfolio attention models cross-item interactions without breaking permutation invariance in P_t.

B Simulation and graph construction

Simulation. For controlled experiments, we generate synthetic labels via the following data-generating process:

$$y_{b,t} = \underbrace{\pi^{\text{ref}}_{b,t}}_{\text{baseline}} + \underbrace{\beta_{\text{size}} \cdot \log(\widetilde{1 + \text{size}_b})}_{\text{size effect}} + \underbrace{\beta_{\text{pf}} \cdot \langle \bar{X}_{P_t \setminus b}, U_{P_t} \rangle}_{\text{portfolio-conditioned term}} + \underbrace{\beta^{\top}_{\text{mkt}} M_t}_{\text{market term}} + \varepsilon_{b,t}, \tag{11}$$

where:

- $\pi^{\text{ref}}_{b,t}$ is the baseline premium (from vendor or deterministic proxy);
- $\widetilde{\log(1+\text{size})}$ is the standardized log-size;
- $\bar{X}_{P_t\setminus b} = \frac{1}{|P_t|-1}\sum_{j\in P_t\setminus\{b\}} X_j$ is the LOO mean of bond feature vectors in the basket;
- $U_{P_t} \in \mathbb{R}^F$ is a portfolio-specific latent direction (sampled once per portfolio);
- $\beta_{\text{pf}} > 0$ controls the strength of the portfolio-conditioned signal (side-agnostic drag);
- $\varepsilon_{b,t} \sim \mathcal{N}(0, \sigma_\varepsilon^2)$ is idiosyncratic noise.

The inner product $\langle \bar{X}_{P_t\setminus b}, U_{P_t}\rangle$ creates a *true* portfolio-conditioned signal that the model must learn to recover.

Cotrade edges. For each eligible (portfolio, date) group we accumulate a pair score

$$s_{ij} = \sum\nolimits_{\text{group}} \underbrace{e^{-\Delta d/30}}_{\text{recency}} \cdot \underbrace{\text{sign}_i \text{sign}_j}_{\text{co vs cross}} \cdot \underbrace{\frac{\min(|\text{DV01}_i|, |\text{DV01}_j|)}{\text{exposure scale}}}_{\text{exposure}} \cdot \underbrace{\text{sim}_{ij}}_{\text{sector/rating/tenor}},$$

then threshold $|s_{ij}|$ by quantile, keep per-node Top-K, and relabel: **COTRADE_CO** if $s_{ij} > 0$, **COTRADE_X** otherwise; weights are $|s_{ij}|$.

Correlation/MI priors and where MV applies them. From daily surprises we compute (i) |corr| and (ii) plug in MI after quantile discretization, on global and rolling windows; we threshold and keep per node Top K. In MV-DGT the relation weights modulate attention logits inside each view; differential fusion aligns their contribution to the structural view.

C Additional figures

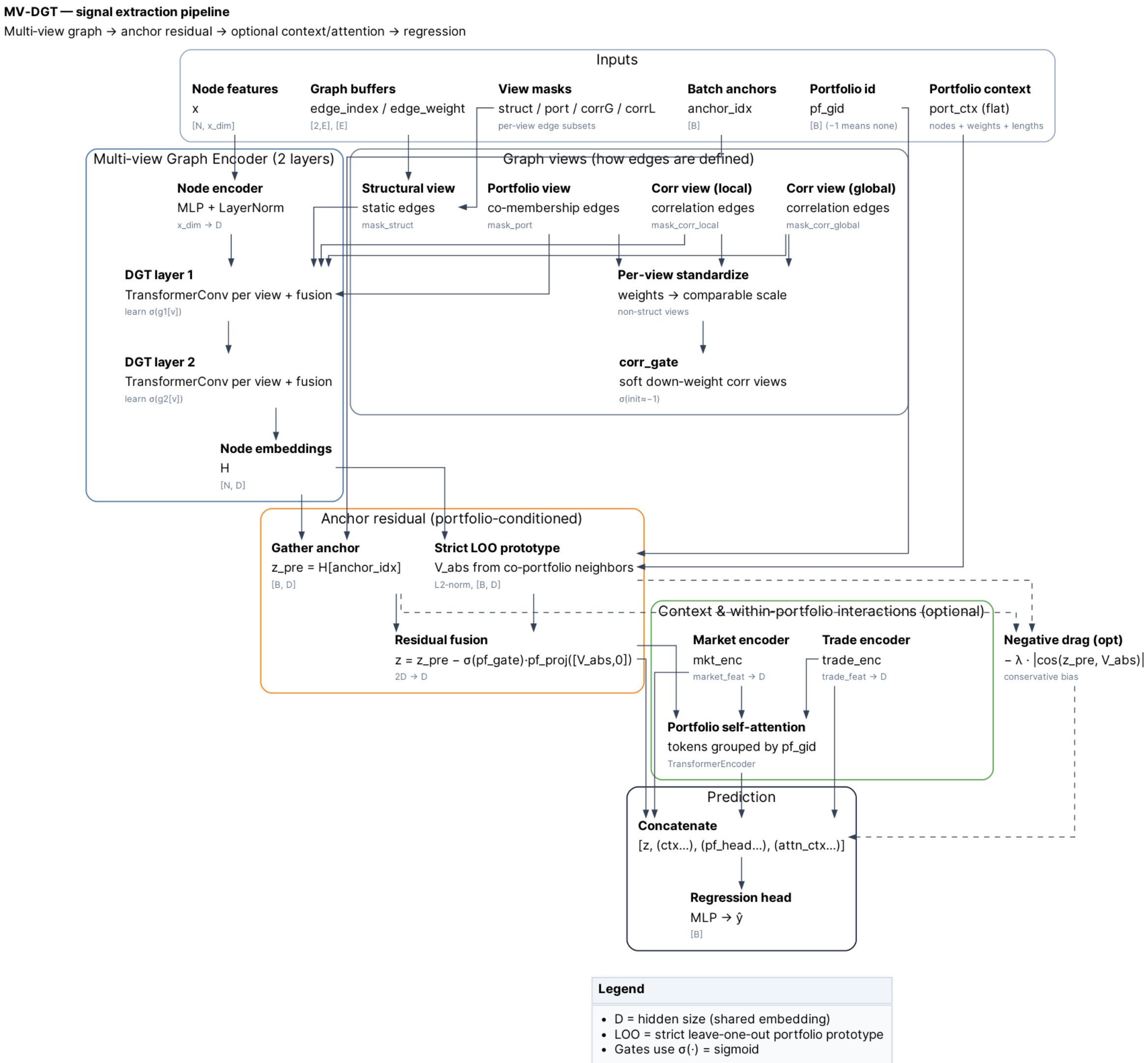


Figure 7: **Signal extraction view (conceptual).** A compact view of how MV-DGT separates sources of signal: structural relations, portfolio/co-trade links, and correlation priors are processed in parallel and combined through differential fusion. Shown for intuition; the concrete featurization and training/serving details follow the same decomposition into views and masks.

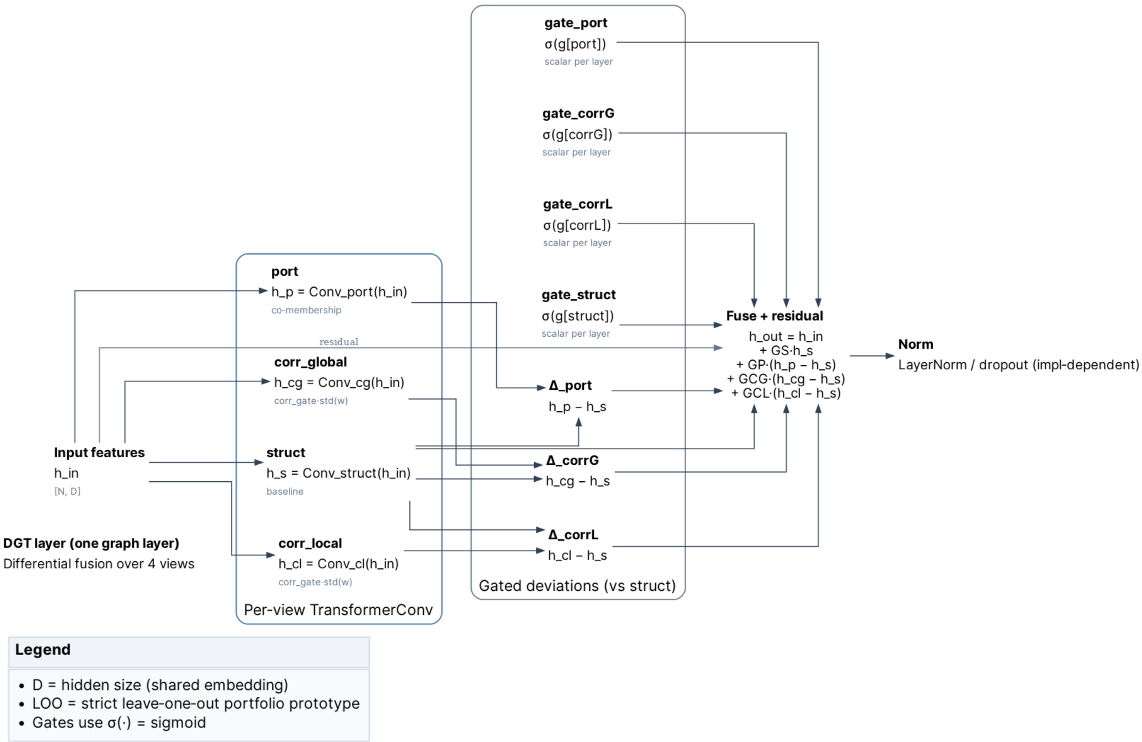


Figure 8: **Differential fusion layer (conceptual).** Structural message passing acts as a baseline, while other views (portfolio and correlation) contribute only their *deviations* from structural via learnable gates. This reduces scale mismatch across relation types and makes the contribution of each view more stable and interpretable.

D Disclaimer

This research prototype and the accompanying code/architecture are provided *as is* for academic evaluation only. They do not constitute investment advice and should not be used directly for trading or risk without independent validation, monitoring, and controls. No warranty is expressed or implied; the author(s) assume no responsibility for any production use or consequential losses.

E Code Repository & Reproducibility

The code for this project is available on GitHub: `https://github.com/olegroshka/pt-liqadj`. To reproduce the paper figures end-to-end (paths as in our run layout), execute:

```
ptliq-paper make-data --root paper_runs/exp001
ptliq-paper score-scenarios --run-dir paper_runs/exp001/models/dgt --out paper/tables
ptliq-paper make-figures --tables-dir paper/tables --out paper/figs
```

References

[1] Hamilton, W., Ying, Z., Leskovec, J. (2017). Inductive Representation Learning on Large Graphs. *NeurIPS*.

[2] Veličković, P., et al. (2018). Graph Attention Networks. *ICLR*.

[3] Vaswani, A., et al. (2017). Attention Is All You Need. *NeurIPS*.

[4] Zaheer, M., et al. (2017). Deep Sets. *NeurIPS*.

[5] Dwivedi, V. P., Bresson, X. (2021). A Generalization of Transformer Networks to Graphs. *AAAI Workshops*.

[6] Bouchaud, J. P., Farmer, J. D., Lillo, F. (2010). How markets slowly digest changes in supply and demand. In *Handbook of Financial Markets*.

[7] Gatheral, J. (2010). No-dynamic-arbitrage and market impact. *Quantitative Finance*.

[8] Cover, T. M., Thomas, J. A. (2006). *Elements of Information Theory*. Wiley.

[9] Stanford CS224W (2021). *Stock Market Forecasting with Differential Graph Transformer*. Blog article. `https://medium.com/stanford-cs224w/stock-market-forecasting-with-differential-graph-transformer-62d095ebc821`